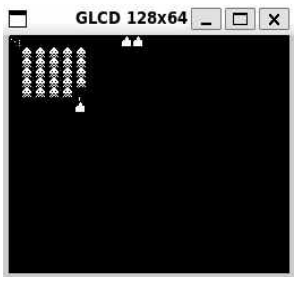
오픈소스 기반 반도체 칩 종합 설계

권효재

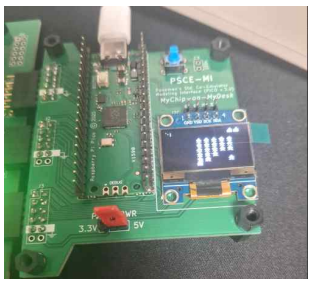
이번에는 지난번에 진행했던 Q13과 Q14를 실제 FPGA 보드에서 동작시켜 보았다. FPGA 에뮬레이션 키트를 사용해서 시뮬레이션뿐만 아니라 실제 OLED 화면에서도 게임이 실행되는지 확인했다. 추가로 아두이노로 만든 미니 오실로스코프를 사용해서 라즈베리파이에서 출력되는 클럭도 측정해보았다.

- Q13 FPGA 실습

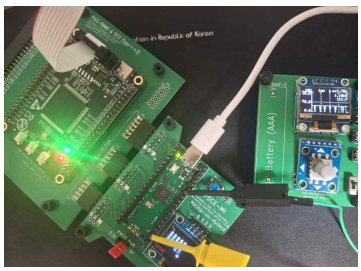
먼저 Q13의 simulation 디렉터리에서 4_space_invaders_engine_glcd.v.Ok를 선택해 시뮬레이션을 진행했다. 지난번에는 빌드와 코드 비교만 했었는데, 이번에는 창에서 우주 침략자 게임이 실행되는 것까지 확인할 수 있었다. 곧바로 emulation 디렉터리에서 FPGA와 라즈베리파이 펌웨어, 테스트벤치 빌드를 진행했다. FPGA에 파일을 업로드할 때 문제가 생겼는데 openFPGAloader에서 FPGA의 IDCODE를 인식하지 못했다. 확인해보니 전에 설치했던 openFPGAloader와 실행되는 버전이 달랐던 것이 문제였고,1.1.1 버전을 따로 설치해서 실행해보니 Load SRAM 100%라고 뜨면서 정상적으로 완료됐다. 라즈베리파이의 경우에는 윈도우에서 USB 장치가 연결된 것으로 나타나도 WSL에서 바로 사용할 수 없었다. powershell에서 usbipd list로 장치를 확인하고 우분투에 연결하는 과정을 거쳤는데, attach를 계속 반복해야했다. 보드를 다시 뺐다가 연결하면 이 과정도 다시 확인해야 했다. 이후 에뮬레이션을 실행해보니 FPGA와 연결되었다는 메시지가 나타났고, Frame 값도 증가했다. 다만 프레임이 상당히 느리게 진행되어 OLED 화면도 늦게 갱신됐다. 다음으로 application 디렉터리에서 프로토타이핑을 진행했다. 키보드의 좌우 방향키와 스페이스바를 입력하면 OLED에서도 포대 이동과 발사를 확인할 수 있었다. 하지만 키보드 입력에 대한 반응이 느려서 실제 게임처럼 조작하기는 어려웠다.



Q13 에뮬레이션



Q13 에뮬레이션



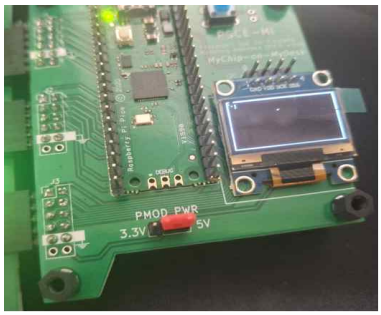
오실로스코프 측정

과제 13 의 미니 오실로스코프 측정도 같이 진행했는데 피코보드의의 GPIO 28핀에서 FPGA로 전달되는 클락을 측정해야 했다. 그런데 오실로스코프를 연결하는 과정에서 아두이노의 GND선을 피코의 핀에 잘못 연결해서 보드들의 전원이 모두 꺼지는 문제가 발생했다. 처음에는 보드들의 불이 전부 꺼지질래 싹 고장 난 줄 알고 크게 절망했지만, 다행히 연결된 선들을 분리하고 시간이 지난 뒤 다시 연결해보니 정상적으로 작동했다. 이후 회로도를 다시 확인하고 핀 위치를 맞춰 연결했다. 라즈베리파이 피코 보드의 회로도와 아두이노 보드의 회로도를 다시한번 꼼꼼히 확인하고, 핀 개수도 잘 세서 또 단락되는 일 없도록 조심하고 또 조심했다.

미니 오실로스코프로 피코의 클럭 출력을 측정해보니 평균 전압은 약 1.55V였고, 주파수는 20Hz~2.5kHz 범위에서 나타났다. 샘플 간격을 변경하니 측정되는 주파수도 달라졌는데, 100kHz 클럭은 한 주기가 10µs이므로 샘플 간격이 길어지면 신호변화를 제대로 측정하기 어렵다. 그래서 샘플 간격을 1µs까지 줄여봤지만 unsync가 뜨면서 정상적으로 측정되지 않았다. 처음에는 샘플 간격을 줄이면 측정할 수 있을 거라고 생각했는데, 학습노트를 다시 확인해보니 아두이노에 사용된 ATmega328P의 ADC 최대 샘플링 속도는 약 9.6kHz였다. 따라서 100kHz 클럭을 측정하기에는 오실로스코프 자체의 성능이 부족했던 것 같다.

- Q14 FPGA 실습

Q14는 칩의 면적과 핀 수를 줄이기 위해 개선한 RTL을 사용하는 과정이다. 이전주에 코드를 비교하면서 곱셈을 시프트 연산으로 수정하고, 반복문을 줄인것을 확인했었는데 이번에는 이 RTL로 시뮬레이션과 FPGA 검증을 진행했다. SDL2 시뮬레이션에서는 게임이 정상적으로 실행됐고, FPGA 합성과 업로드도 완료했다. 피코 펌웨어를 업로드하고 에뮬레이션을 실행했을 때도 연결 메시지와 프레임 진행을 확인할 수 있었다. 하지만 실제 OLED에서는 게임 화면이 정상적으로 나타나지 않았다. 테두리와 일부 점만 표시되었고 키보드를 입력해도 정상적인 게임 화면으로 바뀌지 않았다. Q13에서는 같은 보드로 게임이 실행되었기 때문에 OLED 자체의 문제는 아닌 것처럼 보였는데 Q14에서 대체 어떤 문제인지, 출력 방식에 문제가 있는 것인지 추가로 확인이 필요했다. 현재까지는 시뮬레이션과 FPGA 연결은 성공했지만, 실제 OLED 게임 동작은 확인하지 못했다.



Q14 OLED 출력 결과

- 버튼 입력과 화면 갱신 시점

화면을 그리는 중간에 버튼 입력으로 위치가 바뀌면 화면이 이상하게 출력될 수 있기 때문에 화면 갱신이 끝나고 나서 버튼 입력을 확인해야 한다고 생각한다. 이전에 Q11 실습에서 키보드 입력에 지나치게 빠르게 반응하는 문제를 확인했었다. 이번 Q13에서는 chk_btn을 추가해서 화면을 그리는 중간에 포대의 위치가 바뀌지 않도록 화면 마지막 행까지 출력하고 버튼 입력을 확인하도록 수정한 것을 볼 수 있었다.

- V_Sync가 필요한 이유

Vsync는 화면의 한 프레임이 끝나고 다음 프레임이 시작되는 시점을 알려주는 신호인데 화면은 픽셀을 순서대로 출력해서 만들어지고, 이 프레임의 시작과 끝을 구분하지 못하면 화면을 갱신하는 시점도 맞추기 어렵다. 따라서 Vsync를 이용하면 화면을 출력하는 시점과 포대나 외계인의 위치가 바뀌는 시점을 맞출 수 있다고 생각한다.

- FPGA 에뮬레이션이 필요한 이유

시뮬레이션은 컴퓨터에서 RTL의 동작을 확인하는 과정이고, FPGA 에뮬레이션은 RTL을 실제 FPGA에 올려서 동작을 확인하는 과정이다. 시뮬레이션에서는 내부 신호를 쉽게 확인할 수 있지만 실제 보드의 연결 상태나 입력출력 동작까지 모두 확인할 수 있는 건 아니다. 이번 실습에서 Q13과 Q14 모두 시뮬레이션은 정상적으로 실행되었지만 실제 OLED의 출력 결과는 달랐다. 따라서 시뮬레이션이 성공했다고 하더라도 실제 하드웨어에서도 정상적으로 작동하는지 확인하는 과정이 필요하다고 생각한다.

- 상용 하드웨어 에뮬레이터의 내부 신호 추적

FPGA 에뮬레이션에서는 시뮬레이션처럼 파형을 보고 내부 신호를 모두 확인하기 어렵다는 단점이 있다. 비싼 툴들인 Synopsys의 ZeBu와 Cadence의 Palladium에서는 필요한 신호를 선택해서 저장하거나 문제가 발생한 시점의 동작을 다시 분석하는 방식으로 내부 신호를 기록하고 다시 확인할 수 있는 기능을 제공해서 이런 문제들을 해결한다고 한다. 기록한 신호를 파형으로 확인할 수도 있다.

-FPGA 핀 할당과 펌웨어 비교

(a) FPGA 부품명 및 입출력 핀 할당 비교

Quartus 합성 스크립트와 회로도를 같이 살펴보았다. 회로도에는 Cyclone IV EP4CE6이라고 적혀 있었고, tcl 파일에는 EP4CE6E22C8로 되어 있었다. J8 커넥터 부분을 보니 신호 이름 뒤에 숫자들이 붙어 있었는데, lcd_x[0]의 142번과 btn_fire의 111번이 tcl의 핀 번호와 일치했다. 처음에는 이 숫자들이 뭘 의미하는지 몰랐는데, tcl 파일과 비교해보니 FPGA의 핀 번호를 나타내는 것 같았다.

(b) 프로토타이핑과 에뮬레이션 펌웨어 비교

두 펌웨어를 비교해보니 프로토타이핑에서는 RP2040_PWM으로 100kHz 클럭을 만들고, Serial.read()를 이용해 PC의 키보드 입력을 받고 있었다. 에뮬레이션에서는 PSCE 라이브러리와 RxPacket(), TxPacket()이라는 함수를 사용하고 있었는데, 테스트벤치와 FPGA 사이에서 데이터를 주고받기 위한 것 같았다. 코드 전체를 이해하기는 어려웠지만 프로토타이핑과 에뮬레이션에서 클럭과 입력을 처리하는 방식이 다르다는 점은 확인할 수 있었다.

이번 실습에서는 Q13의 시뮬레이션부터 FPGA 에뮬레이션, 프로토타이핑까지 진행했고 OLED에서도 게임이 작동하는 것을 확인했다. 에뮬레이션은 굉장히 느리게 진행되었고, Q14의 경우 시뮬레이션과 FPGA 연결까지 성공했지만 OLED에는 정상적인 게임 화면이 나타나지 않았다. 실습 중간에 USB 연결이나 FPGA 업로드 오류도 여러 번 발생해서 해결하는 데 시간도 꽤 걸렸다. 지난주에 시뮬레이션과 에뮬레이션의 차이를 정리했었는데, 이번에는 실제 보드를 사용하면서 프로그램 빌드뿐만 아니라 FPGA 설정이나 라즈베리파이 연결 상태 등 확인해야 할 부분이 많았다.